Connections to Statistics

From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

Statistics in the Story

Machine learning is applied statistics at scale. The concepts your students encounter in an introductory statistics course — regression, residuals, probability, hypothesis testing — are not just analogies for what neural networks do. They are the actual mathematical framework the field was built on.

Statistics concept	**AI counterpart**
Linear regression $\widehat{y} = mx + b$	Forward pass of a single neuron
Residual $\widehat{y}_i - y_i$	Prediction error, drives learning
Least squares criterion	Mean squared error loss function
Minimizing SSE	Gradient descent on MSE
Logistic regression	Single neuron with sigmoid activation
Probability as model output	Softmax output layer
Maximum likelihood estimation	Cross-entropy loss function
Confusion matrix, accuracy	Classifier evaluation
Precision, recall	Cost-sensitive evaluation
Generalization from sample	Train/test split, overfitting

Linear Regression: The Direct Connection

Students in Math 146 learn that a regression line $\widehat{y} = b_0 + b_1 x$ is found by minimizing the sum of squared residuals. This is exactly the loss function used to train a neural network's output layer for continuous prediction.

The residual $\widehat{y}_i - y_i$ measures how wrong the model is for one data point. The mean squared error averages this over all n training examples:

$$L = \frac{1}{n} \sum_{i=1}^{n} (\widehat{y}_i - y_i)^2$$

Minimizing this by taking derivatives and setting them to zero — the approach Math 146 uses for the two-parameter line — gives the normal equations. For neu-

ral networks with millions of parameters, the normal equations are computationally infeasible, so gradient descent is used instead. But the objective is identical: minimize the average squared residual.

Key result. The regression line your students find in Math 146 minimizes the same loss function that trains a neural network. Least squares regression is gradient descent on MSE, solved exactly rather than iteratively. Every student who has minimized SSE has worked with the foundation of machine learning.

Historical note. The method of least squares (Gauss, Legendre, early 19th century) predates neural networks by 150 years. When Rosenblatt designed the Perceptron learning rule in 1958, he was explicitly extending the least squares framework to a nonlinear model. The field of machine learning grew directly from the statistical tradition of regression, not from computer science. The split between "statistics" and "machine learning" is largely sociological, not mathematical.

From Regression to Classification

Linear regression predicts a continuous value. When the outcome is categorical — spam or not spam, malignant or benign — a different model is needed. Logistic regression extends the linear model by passing the linear combination through the sigmoid function:

$$\hat{y} = \sigma(b_0 + b_1 x) = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$

The output is a probability: the model's estimated probability that the input belongs to class 1. A *decision threshold* (typically 0.5) converts the probability to a class prediction:

$$\text{predicted class} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

A single neuron with sigmoid activation and a single input feature *is* logistic regression. The neural network story begins here.

Common misconception. Logistic regression, despite its name, is a classification model, not a regression model. The "regression" refers to the underlying linear combination, not the type of output. This naming confusion is historical and unfortunately persistent.

Evaluating a Classifier

Why Accuracy Is Not Enough

The simplest evaluation metric for a binary classifier is accuracy: the fraction of predictions that are correct. But accuracy alone is misleading when the classes are imbalanced.

Consider a medical screening test for a condition that affects 1% of the population. A classifier that always predicts "negative'' achieves 99% accuracy — while being completely useless for detecting the condition it was designed to find.

The right tools for evaluating a classifier are built from the *confusion matrix*.

The Confusion Matrix

For a binary classifier with classes Positive (P) and Negative (N):

		Predicted	
		Positive	Negative
Actual	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

From these four counts, we define:

Metric	**Formula**	**Answers**
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	What fraction did we get right?
Precision	$\frac{TP}{TP+FP}$	When we predict positive, how often are we right?
Recall	$\frac{TP}{TP+FN}$	Of all actual positives, how many did we find?
F1 score	$\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$	Harmonic mean of precision and recall

Precision and recall trade off against each other. Lowering the decision threshold (predicting positive more freely) increases recall but decreases precision. The right balance depends on the cost of each type of error.

Connection to hypothesis testing. A False Positive is a Type I error: rejecting a true null hypothesis (incorrectly predicting positive when the true class is negative). A False Negative is a Type II error: failing to reject a false null hypothesis (missing a true positive). The decision threshold in a classifier plays the same role as the significance level α in hypothesis testing: it controls the trade-off between these two error types. Adjusting the threshold is equivalent to adjusting α.

Student Activity: Evaluating a Binary Classifier

Background

A binary classifier takes an input and outputs a probability score between 0 and 1. A decision threshold converts the score to a class prediction: scores above the threshold are predicted Positive, scores below are predicted Negative.

In this activity you will evaluate a classifier using a confusion matrix, compute standard metrics, and explore what happens when the threshold changes.

The Data

A spam filter has been tested on 20 emails. The table below shows the classifier's probability score and the true label (1 = spam, 0 = not spam) for each email.

Email	Score	True	Email	Score	True
1	0.92	1	11	0.45	0
2	0.87	1	12	0.41	1
3	0.81	1	13	0.38	0
4	0.76	0	14	0.35	0
5	0.74	1	15	0.29	0
6	0.68	1	16	0.24	1
7	0.63	0	17	0.21	0
8	0.59	1	18	0.15	0
9	0.55	0	19	0.09	0
10	0.51	1	20	0.04	0

There are **9 actual spam emails** (true label = 1) and **11 actual non-spam emails** (true label = 0).

Part 1: Threshold = 0.5

Using a decision threshold of 0.5, predict Positive (spam) for any email with score ≥ 0.5 and Negative (not spam) for scores < 0.5.

Step 1. For each email, write P (predicted positive) or N (predicted negative) in the last column of the table above.

Step 2. Fill in the confusion matrix:

		Predicted	
		Spam	Not Spam
Actual	Spam	TP = _____	FN = _____
	Not Spam	FP = _____	TN = _____

Step 3. Compute the metrics:

Accuracy = _____ Precision = _____ Recall = _____

Part 2: Lower the Threshold to 0.3

Now predict Positive for any email with score ≥ 0.3.

Step 1. Fill in the new confusion matrix:

		Predicted	
		Spam	Not Spam
Actual	Spam	TP = _____	FN = _____
	Not Spam	FP = _____	TN = _____

Step 2. Compute the metrics:

Accuracy = _____ Precision = _____ Recall = _____

Part 3: Reflection

1. When the threshold decreased from 0.5 to 0.3, what happened to recall? What happened to precision?

2. This classifier is a spam filter. Which error is more costly to a user: a False Positive (legitimate email flagged as spam) or a False Negative (spam email reaching the inbox)? Which threshold would you choose, and why?

3. Now imagine this classifier is a cancer screening tool, where Positive means "further testing recommended.'' Which error is more costly? Would you use a higher or lower threshold than you chose for the spam filter?

The design insight. Choosing a decision threshold is a design decision, not a mathematical one. It encodes a judgment about the relative cost of False Positives and False Negatives. In a production AI system, this judgment is made by engineers and domain experts, not by the training algorithm. The same model can behave very differently depending on where the threshold is

set.

Answer Key

Part 1 (threshold = 0.5):

Predicted Positive (score ≥ 0.5): emails 1–10 (scores 0.51–0.92). Predicted Negative (score < 0.5): emails 11–20.

Actual spam emails: 1, 2, 3, 5, 6, 8, 10, 12, 16. Actual non-spam emails: 4, 7, 9, 11, 13, 14, 15, 17, 18, 19, 20.

		Predicted	
		Spam	Not Spam
Actual	Spam	TP = 7	FN = 2
	Not Spam	FP = 3	TN = 8

(Emails 1,2,3,5,6,8,10 correctly flagged as spam = 7 TP. Emails 12,16 are spam but scored below 0.5 = 2 FN. Emails 4,7,9 are not spam but scored ≥ 0.5 = 3 FP. Emails 11,13,14,15,17,18,19,20 correctly not flagged = 8 TN.)

$$\text{Accuracy} = \frac{7+8}{20} = \frac{15}{20} = 0.75 \qquad \text{Precision} = \frac{7}{7+3} = 0.70 \qquad \text{Recall} = \frac{7}{7+2} \approx 0.78$$

Part 2 (threshold = 0.3):

Predicted Positive (score ≥ 0.3): emails 1–14 (scores 0.35–0.92). Predicted Negative: emails 15–20.

Actual spam in emails 1–14: 1,2,3,5,6,8,10,12 = 8 TP. Actual spam in emails 15–20: 16 = 1 FN. Actual non-spam in emails 1–14: 4,7,9,11,13,14 = 6 FP. Actual non-spam in emails 15–20: 15,17,18,19,20 = 5 TN.

		Predicted	
		Spam	Not Spam
Actual	Spam	TP = 8	FN = 1
	Not Spam	FP = 6	TN = 5

$$\text{Accuracy} = \frac{8+5}{20} = \frac{13}{20} = 0.65 \qquad \text{Precision} = \frac{8}{8+6} \approx 0.57 \qquad \text{Recall} = \frac{8}{8+1} \approx 0.89$$

Part 3 discussion notes:

Lowering the threshold increased recall (from 0.78 to 0.89) but decreased both precision (0.70 to 0.53) and accuracy (0.75 to 0.60). The model catches more spam but flags more legitimate email.

For a spam filter, a False Positive (losing legitimate email) is typically more costly than a False Negative (receiving spam), so a higher threshold is usually preferred.

For cancer screening, a False Negative (missing a cancer case) is far more costly than a False Positive (recommending unnecessary follow-up testing), so a lower threshold is appropriate. This is why medical screening tools are tuned for high recall even at the cost of precision.

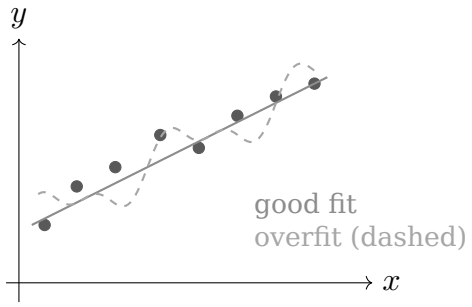
Overfitting and Generalization: A Note for Instructors

The following section is intended as background for statistics instructors. The concepts are important for understanding how and why machine learning models sometimes fail, and connect directly to statistical ideas about inference from samples. How much of this to share with students is a matter of instructor judgment.

The Core Problem

A model trained on data faces the same fundamental challenge as any statistical inference: it must generalize from a sample to a population. In statistics, we use a sample to estimate population parameters, and we worry about whether our estimates are reliable. In machine learning, we use training data to fit model parameters, and we worry about whether the model will perform well on new, unseen data.

A model that performs perfectly on its training data but poorly on new data is *overfit*. It has learned the training data — including its noise and idiosyncrasies — rather than the underlying pattern.



The overfit model has lower training error but higher error on new data. It has confused noise for signal.

The Train/Test Split

The standard remedy is to divide the available data into a *training set* and a *test set* before fitting the model. The model is fit on the training set only. Performance is evaluated on the test set, which the model has never seen.

A typical split is 80% training / 20% test, though this varies with dataset size. The test set error is an unbiased estimate of how the model will perform in deployment.

The statistical parallel. The train/test split is conceptually identical to the distinction between an estimation sample and a validation sample in statistics. The test set plays the role of a held-out population: we use the training data to fit parameters, then ask how well those parameters generalize. A model with low training error and high test error has overfit — it has found parameter values that describe the training sample well but the population poorly. This is

the machine learning version of overfitting a polynomial regression to a small sample.

Bias and Variance

Overfit models have *high variance*: small changes in training data produce large changes in the model. Underfit models (too simple for the data) have *high bias*: systematic errors that persist regardless of training data. This is the *bias-variance tradeoff*, a central concept in both statistics and machine learning.

In statistics, your students see this when choosing between a simple linear model and a higher-degree polynomial: the polynomial fits training data better (lower bias) but is more sensitive to which data points happen to be in the sample (higher variance).

For instructors. The concepts of generalization, overfitting, train/test split, and bias-variance tradeoff are directly accessible to students who have seen regression in Math 146. The framing "does this model describe the sample or the population?" connects immediately to everything students already know about statistical inference. These ideas explain why AI systems sometimes fail in deployment despite performing well during development — a real and important phenomenon your students will encounter as citizens and professionals.

Classroom Connections: Math 146 Topic Map

Descriptive Statistics and Regression

Math 146 students compute means, standard deviations, and regression lines. Each of these connects directly:

The **mean** as the minimizer of squared deviations ($\bar{x} = \arg\min_c \sum (x_i - c)^2$) is the same principle as MSE loss minimization. The mean is the simplest possible model — a neural network with zero parameters, predicting the same value for every input.

The **regression line** minimizes the sum of squared residuals. This is exactly MSE gradient descent, solved in closed form. When you teach students to find b_0 and b_1, you are teaching them the objective function of machine learning.

The **residual plot** is a diagnostic for model fit. In machine learning, the loss curve (training loss vs. epoch) plays the same role: we watch it to diagnose whether the model is learning, stuck, or overfit.

Probability

Math 146 introduces probability as a number between 0 and 1 describing likelihood. Neural network classifiers output exactly this: a probability that the input belongs

to a given class. The softmax function (the multiclass generalization of sigmoid) produces a valid probability distribution over all classes — non-negative, summing to 1 — directly interpretable as the model's belief.

Cross-entropy loss is a measure of how good a set of predicted probabilities is. When the model predicts high probability for the correct class, the loss is low. Students who have seen $P(A)$ and $-\log P(A)$ (log-likelihood) have the tools to understand why cross-entropy is the natural loss for probabilistic classifiers.

A Framing for Students

For instructors. The most direct bridge from Math 146 to AI is this: "A neural network is a nonlinear regression model. It finds parameters that minimize prediction error on training data, outputs probabilities for classification problems, and is evaluated using the same tools — residuals, accuracy, precision, recall — that statisticians have used for decades. The only thing that changed is the scale: from two parameters to 175 billion, and from normal equations to gradient descent." This framing requires no new mathematics and connects immediately to what your students already know.